

Cours de Programmation en C – EI-2I

Travaux Dirigés

TD1

Exercice 1 – Post et Pré-incrémentation

On considère le code suivant

```
int i = 5, j, k, l, m;  
j = i++;  
k = ++i;  
l = i--;  
m = --i + l++;
```

Quels sont les valeurs des variables *i*, *j*, *k*, *l* et *m* après l'exécution de cette portion de programme ?

Solution:

- *j* = *i*++; implique que *j* vaut 5, et qu'ensuite *i* passe à 6
- *k* = ++*i*; implique que *i* passe à 7 et qu'ensuite *k* prenne la valeur 7
- *l* = *i*++; implique *l* vaut 7, et qu'en suite *i* passe à 6
- enfin, *m* = --*i* + *l*++; implique que *i* passe à 5, que *m* vaut donc 12, et qu'enfin *l* passe à 8

Exercice 2 – Comparaisons

```
int a=2, b=3, c=5;  
int x = ((c++ >= a) < (b % c)) + (c>a);
```

Que vaut *x* ?

Solution:

(*c*++ >= *a*) est évalué et vaut 1, (*b* % *c*) vaut 3, donc ((*c*++ >= *a*) < (*b* % *c*)) vaut 1. De plus (*c*>*a*) est vrai donc vaut 1, et ainsi *x* prend pour valeur 2.

Exercice 3 – Décalages et opérations bit-à-bit

- Diviser (en entier) *x* par 256
- Trouver la valeur du bit n° *i* de *x*
- Mettre à 0 les bits {1,4,5} de *x* et mettre à 1 ses bits {0,2}
- Mettre à 1 le bit n° *i* de *x*
- Mettre à 0 le bit n° *i* de *x*
- Indiquer si les bits 4 et 6 de *x* et de *y* sont égaux
- Indiquer si au moins un des bits 1, 4 et 6 de *x* et de *y* sont égaux

Solution:

- $256 = 2^8$, donc diviser x par 256 revient à décaler x de 8 bits à droite : $x >> 8$
- On peut aussi décaler de i bits vers la droite pour que le bit n° i se trouve en position 0; et ensuite masquer pour ne récupérer que ce bit : $(x >> i) \& 1$
- le nombre binaire 11001101 vaut 0xCD ou encore 205. Si on masque (et bit-à-bit) x par cette valeur, il ne restera que les bits 7,6 3 2 et 0, les autres (1,4 et 5) auront été mis à 0 : $x \&= 0xCD$;
Ensuite, un *ou* bit-à-bit avec le nombre binaire 00000101 permet de mettre les bits 0 et 2 à 1 : $x |= 5$;
- Il faut réaliser le *et* bit-à-bit, mais avec un masque construit à partir de i . $1 << i$ construit un nombre où seul le $i^{\text{ème}}$ bit vaut 1. Ainsi, si on écrit $x |= 1 << i$; , on met bien le bit n° i de x à 1
- Idem, mais avec un masque contenant que des 1, sauf le $i^{\text{ème}}$ bit, et un *et* bit-à-bit. D'où $x \&= (1 << i)$
- on utilise un masque pour récupérer uniquement les bits 4 et 6, et on compare le résultat : $(x \& 0x50) == (y \& 0x50)$
- un *ou exclusif* bit-à-bit entre x et y permet de savoir quels bits de x et de y sont égaux. Ensuite, il n'y a plus qu'à masquer pour ne récupérer que les bits 1, 4 et 6. Enfin, on peut directement utiliser cette valeur comme valeur booléenne, car elle vaudra 0 si aucun des 3 bits ne sont en commun, et une autre valeur sinon : $(x \wedge y) \& 0x52$

Exercice 4 – Opérateur conditionnel

En utilisant un opérateur conditionnel tertiaire :

- Trouver le maximum de 2 nombres a et b . Le minimum.
- Donner la valeur absolue d'une variable x

Solution:

- $(a > b) ? a : b$ pour le maximum. Et $(a > b) ? b : a$ pour le minimum
- $(a > 0) ? a : -a$

Exercice 5 – Affectation composée

Qu'obtient-on avec le code suivant:

```
int a=1;
a *= a++;
a <=&= 2;
a |= 36;
a %= a++ - 8;
```

Et si a est initialisé à 2 ?

Solution:

Pour $a=1$:

```
a *= a++;      \* a=2 */
a <=&= 2;      \* a=8 */
a |= 36;      \* a=44 */
a %= a++ - 8;  \* a=9 */
```

Pour $a=2$:

```
a *= a++;      \* a=5 */
a <=&= 2;      \* a=20 */
a |= 36;      \* a=52 */
a %= a++ - 8;  \* a=9 */
```

Exercice 6 – Choix selon

Écrire un code C permettant de déterminer un entier y en fonction d'un entier x suivant la correspondance

x	y
12	1
4	2
5	3
42	4
8	5
autre	0

1. avec des `if`
2. avec un `switch ... case`
3. avec un `switch ... case` mais sans utiliser de `break`

Solution:

1.

```
if (x==12)
    y=1;
else if (x==4)
    y=2;
else if (x==5)
    y=3;
else if (x==42)
    y=4;
else if (x==8)
    y=5;
else
    y=0;
```
2.

```
switch (x)
{
    case 12: y=1; break;
    case 4: y=2; break;
    case 5: y=3; break;
    case 42: y=4; break;
    case 8: y=5; break;
    default: y=0;
}
```
3.

```
y=0;
switch (x)
{
    case 8: y++;
    case 42: y++;
    case 5: y++;
    case 4: y++;
    case 12: y++;
}
```

Bien entendu, on n'utilisera jamais la solution n° 3 qui n'est pas lisible (bien que pouvant être plus performante car effectuant moins de branchements).